Makerere University
College of Computing and Information Sciences

Embedded Systems: CSC, 2026/2027

Smart Control and Monitoring System for CoCIS

System Operation Report

ESP32-S3 · ESP-IDF v6.0 · FreeRTOS · Wokwi

Group 28

#	Full name	Student Number	Registration Number
1	MUJUNI INNOCENT	2400707155	24/U/07155/PS
2	MARTHA NAMUYIMBWA	2400709436	24/U/09436/PS
3	OCHEN FELIX	2500726621	25/U/26621
4	NAKIMULI JOLLYNE FLAVIA	2400700953	24/U/0953
5	TAMARA ALICE NAZZIWA	2500703586	25/U/03596/PS

Wokwi simulation: `https://wokwi.com/projects/479125502548914177`

October 29, 2026

Contents

1 Introduction **3**

1.1 Background and problem . . . 3

1.2 What we built . . . 3

1.3 How this report maps to the required deliverables . . . 3

1.4 Conventions . . . 3

2 Requirements and How They Are Met **3**

3 System Architecture **4**

3.1 Three terminals, one authority . . . 5

3.2 Why one ESP32 board simulates the three terminals . . . 5

3.3 Layered components . . . 6

4 Hardware Components **6**

4.1 Bill of materials (Wokwi parts) . . . 7

4.2 Pin map . . . 7

4.3 Wiring . . . 8

5 Software Design **12**

5.1 Toolchain and project layout . . . 13

5.2 FreeRTOS tasks . . . 13

5.3 Boot sequence . . . 14

5.4 The node bus . . . 14

5.5 RFID reader driver . . . 14

5.6 Pure decision logic (`core_logic`) . . . 15

5.7 Shared state services . . . 15

5.8 Interrupts . . . 15

5.9 Coding principles . . . 16

6 System Operation and Implemented Features **16**

6.1 Access control at the entrance (**R2.1**, **R2.6**) . . . 17

6.2 Occupancy records (**R2.1**) . . . 19

6.3 Automatic lighting in the lecture rooms (**R2.2**) . . . 19

6.4 Configurable access schedule (**R2.6**) . . . 20

6.5 Two-factor admin authentication (**R2.6**) . . . 20

6.6 Data storage on the SD card (**R2.4**) . . . 21

6.7 Serial interface and management menus (**R2.5**) . . . 22
6.8 LCD, buzzer and LED . . . 22
6.9 Emergency, lockdown and node supervision . . . 23

7 Synchronisation and Reliability 23
7.1 Design rules . . . 24
7.2 Shared resources and how each is protected . . . 24
7.3 Coordination between terminals . . . 25

8 Innovation and Additional Features 25
8.1 Security . . . 26
8.2 Efficiency . . . 26
8.3 Reliability and safety . . . 26
8.4 Usefulness to management . . . 26
8.5 Engineering quality . . . 27
8.6 Optional cloud dashboard (prepared extension) . . . 27

9 Testing 27
9.1 Unit tests of the decision logic . . . 28
9.2 Build . . . 28
9.3 Simulation testing in Wokwi . . . 28
9.4 Defects found and fixed . . . 30
9.5 Dashboard . . . 30

10 Limitations and Future Work 30
10.1 Known limitations . . . 31
10.2 Future work . . . 31

11 Group Members, Contributions and Videos 31
11.1 Group member details . . . 32
11.2 Individual contributions . . . 32
11.3 Individual video demonstrations . . . 33

A Serial Command Reference 33

B Building, Running and Demonstrating 34
B.1 Build and run the firmware . . . 35
B.2 Running the Wokwi link . . . 35
B.3 Other tasks (from the repository root) . . . 35
B.4 Demonstration script . . . 35

1 Introduction

1.1 Background and problem

The College of Computing and Information Sciences (CoCIS) keeps valuable equipment that must be protected from unauthorised access, and it pays high electricity bills, partly because lights are left on in empty rooms. In October 2026 management agreed to pilot an automated system on the **main entrance** and **two lecture rooms**.

1.2 What we built

A complete, self-contained control and monitoring system on the ESP32-S3, simulated in Wokwi and written in C with ESP-IDF v6.0 and FreeRTOS:

- **Entrance terminal.** Two MFRC522 RFID readers, one at the entry and one at the exit, each with its own servo door lock; anti-passback; an opening-hours schedule that also holds people inside after hours; an LCD and a buzzer; and two-factor admin login (the admin card at a reader + a PIN).
- **Two lecture-room terminals.** A PIR motion sensor and a light sensor per room drive a dimmable lamp. Lights turn off when the room is empty or when there is enough daylight, and the room reports the energy it used and saved.
- **Storage and management.** Every event is written to the SD card as a *hash-chained* (tamper-evident) log line, and a serial command menu shows who is inside, room status, lighting and the access log and lets an authenticated admin change the schedule, users and overrides.

The system runs entirely offline. An *optional* web dashboard (Next.js + PostgreSQL) was also prepared as an extension (section 8); the system does not depend on it.

1.3 How this report maps to the required deliverables

Deliverable (brief §4)	Where
1. System operation report	Section 2 to section 10: architecture, hardware, software design, operation of every feature, synchronisation, testing
2. Wokwi simulation link	Title page and section 11
3. Individual video demonstrations	Section 11.3
4. Individual contributions	Section 11.2
5. Group member details	Title page and section 11

1.4 Conventions

Requirements from the brief are written **R2.1** … **R2.7** (the brief's section numbers). Paths such as `components/core_logic/access_policy.c` are relative to `firmware/` unless they start with `dashboard/` or `docs/`. Default timings and thresholds quoted in this report are the values in `main/Kconfig.projbuild`; all of them can be changed with `idf.py menuconfig` without editing code.

2 Requirements and How They Are Met

Table 1 traces every requirement in the brief to the feature that meets it and to the section of this report that explains it.

Table 1: Requirements traceability.

Brief	Requirement	Implementation	Report
R2.1	Separate entry and exit points; RFID scan at both	Two MFRC522 RFID readers (entry and exit), each with its own servo door. Card → `access_decide()` → door opens for 3 s.	6.1
R2.1	Verify identity, open the right door	User registry in NVS (UID, name, role, enabled). Only the door on the scanned side opens.	6.1
R2.1	Records of entry and exit; who is inside now	Every scan is an `ACCESS` log line. The occupancy ledger (persisted) answers `who`.	6.1, 6.7
R2.2	Detect room occupancy and control the lights	PIR + 30 s grace timer, 10 s dim warning, daylight cut-off with hysteresis, PWM lamp.	6.3
R2.2	Reduce wasted electricity	Lamp off when vacant or bright. Energy used and saved per room logged every 10 min.	6.3
R2.3	Three terminals (Room 1, Room 2, Entrance)	Three independent terminal programs (hub task, two room-node tasks) that share no variables.	3
R2.3	Effective communication and coordination	Message bus with per-sender sequence numbers, duplicate rejection, heartbeats and offline detection.	5.4, 7
R2.4	SD card storage	One file per day, `/sdcard/LOGS/YYYYMMDD.LOG`, hash-chained, `fsync` after every line, RAM fallback.	6.6
R2.5	Serial menus and commands	15 commands: monitoring (`status`, `who`, `rooms`, `users`, `log`), configuration and simulation.	6.7, App. A
R2.5	Additional output interfaces	20×4 LCD, buzzer patterns, green LED.	6.8
R2.6	Configurable access period	`schedule set HH:MM HH:MM [days]`, overnight windows supported, stored in NVS.	6.4
R2.6	Restrict entry *and exit* outside hours	Ordinary users cannot enter or leave after hours; an alert calls security, who can escort them out. Break-glass overrides everything for safety.	6.1
R2.6	Only authenticated, authorised personnel may change it	Two-factor login: admin card + 4-digit PIN, 5-minute session, lockout after 3 wrong PINs.	6.5
R2.7	Synchronisation; no conflicts, inconsistent records or races	Single-writer hub, storage gatekeeper task, one mutex per shared module, atomic NVS commits, ordered bus.	7
R2.7	Accurate records, reliable coordination	Hash chain + `log verify`; log sequence resumes after reboot; node heartbeats.	6.6, 7
R3	Innovation	Tamper-evident log, daylight harvesting, energy accounting, 2FA, escort window, break-glass, lockdown, generated wiring, host unit tests, optional cloud dashboard.	8

3 System Architecture

3.1 Three terminals, one authority

The brief asks for three control terminals: the main entrance, Lecture Room 1 and Lecture Room 2 (**R2.3**). Each one is a separate program with its own inputs and outputs:

Entrance terminal (the *hub*). Reads the entry and exit RFID readers and the break-glass button, decides who may pass, drives the two door servos, the LCD, the buzzer and the LED, and keeps the occupancy ledger. It is the **single authority**: it is the only code that makes access decisions or changes who is inside.

Room terminals (*room nodes* 1 and 2). Each reads its own PIR and light sensor, decides its own lamp level and reports its state and a heartbeat to the hub. A room keeps controlling its lights even if the hub stops.

The terminals share no variables. They exchange **messages only**, over the node **bus** (section 5.4). Two supporting tasks serve all terminals: the **storage gatekeeper**, the only code that touches the SD card, and the **serial CLI**.

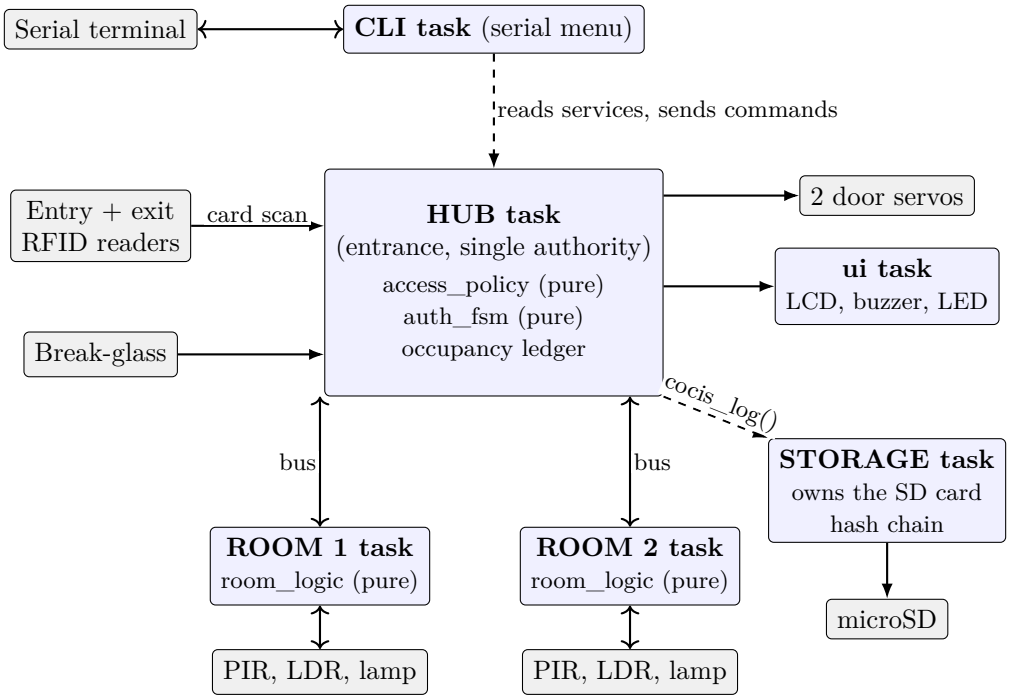


Figure 1: Runtime architecture. Blue boxes are FreeRTOS tasks, grey boxes are hardware. Solid arrows between tasks are bus messages; every task logs through `cocis_log()`, which queues the line to the storage task.

3.2 Why one ESP32 board simulates the three terminals

The brief says each terminal *may* use a separate ESP32 (**R2.3**); it does not require it. We run all three terminals on **one ESP32-S3** in Wokwi, built so that splitting them onto three boards later changes only the transport:

1. **Isolation is real, not cosmetic.** Each terminal is its own FreeRTOS task with its own pins and state. Room nodes never call hub functions and the hub never reads room

variables; everything passes through `bus_send()` / `bus_recv()`.

2. **The bus behaves like a network.** Messages carry a source id and a per-sender sequence number; the receiver drops duplicates and stale frames, the hub detects silent nodes by missing heartbeats, and a room can be "unplugged" with `node 2 offline` to demonstrate failure handling (section 7).
3. **Simulation practicality.** A single Wokwi project shows the whole building at once in one simulator, which makes the demonstration and the shared link reliable.

On three physical boards, the queue transport in `components/bus/bus.c` would be replaced by ESP-NOW or Wi-Fi frames carrying the same `bus_msg_t` structure; the terminal code above it stays the same. This replacement transport is *designed but not implemented* (section 10).

3.3 Layered components

The firmware is split into ESP-IDF components with one concern each. Dependencies only point downward, so lower layers never know about higher ones.

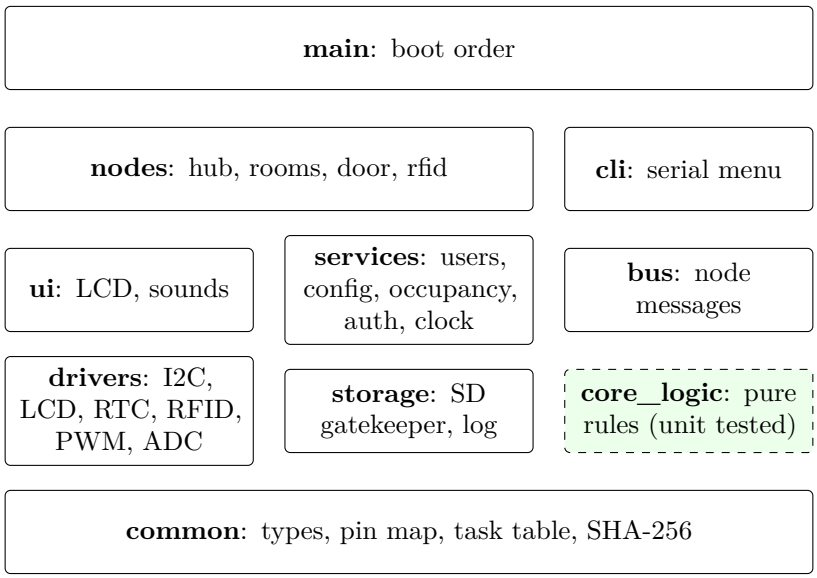


Figure 2: Component layers (`firmware/components/`). The dashed box has no ESP-IDF calls, so it compiles and is tested on a PC.

Component	Responsibility
`common`	Shared enums (declared once with X-macros, which also generate their string tables), the pin map `board_pins.h`, the task table `task_config.h`, SHA-256.
`core_logic`	**Pure decision logic**: access rules, room lighting, admin two-factor state machine. No hardware, no RTOS, so it is unit-tested on a PC.
`drivers`	Thin hardware drivers: I2C bus, LCD2004, DS1307 RTC, MFRC522 RFID reader, buttons, PWM (servo, lamp, buzzer), light sensor.
`bus`	Messages between terminals with sequence numbers and duplicate rejection.
`storage`	The SD card gatekeeper task and the hash-chained log.
`services`	Shared state, each module behind its own mutex and persisted in NVS: users, config (schedule, lockdown), occupancy ledger, auth session, clock.
`ui`	The task that owns the LCD, buzzer and green LED.
`nodes`	The three terminals (hub, two rooms), the door servos and the RFID reader task.
`cli`	The serial management menu (`esp_console`).

4 Hardware Components

4.1 Bill of materials (Wokwi parts)

The circuit is `firmware/diagram.json`: 20 electronic parts plus 14 text labels.

Part (Wokwi type)	Qty	Purpose
ESP32-S3-DevKitC-1	1	Microcontroller: dual-core Xtensa LX7 at 160 MHz, runs all terminals.
Entrance terminal		
MFRC522 RFID reader (`board-mfrc522`)	2	Entry reader and exit reader, 13.56 MHz ISO 14443 A cards, SPI. Wokwi provides the cards (section 4.1).
Servo motor	2	Door locks: entry door and exit door (0° closed, 90° open).
LCD 20×4 (I2C, 0x27)	1	Status display and messages (section 6.8).
Buzzer	1	Audible feedback: granted, denied, information, alarm.
LED (green) + 220 Ω resistor	1	"Access granted" indicator, on for 2 s.
Push button (key `e`)	1	Break-glass emergency: opens both doors.
DS1307 RTC (I2C, 0x68)	1	Real-time clock, so the schedule works without a network.
microSD card (SPI)	1	Event log storage (**R2.4**).
Each lecture room (2 sets)		
PIR motion sensor	1	Occupancy detection.
Photoresistor (LDR) module	1	Ambient light in lux, read on the analog output (ADC1).
LED (white) + resistor	1	The room's lamp, dimmed by PWM.

RFID readers and cards. The entrance has two MFRC522 readers: the **entry reader** (people coming in) and the **exit reader** (people going out), each beside its own door. In Wokwi, clicking a reader opens its control panel: choose a card and press **Tap** (the card is held to the reader for 0.5 s) or **Hold**. The preset cards are registered as the demo users:

Wokwi card	UID	Registered as	Role
Blue	`01:02:03:04`	Alice	STUDENT
Green	`11:22:33:44`	Bob	STAFF
Yellow	`55:66:77:88`	Carol	SECURITY
Red	`AA:BB:CC:DD`	Admin	ADMIN (PIN `1234`)
Key fob	`C0:FF:EE:99`	– (unregistered)	–
NFC tag	`04:11:22:33:44:55:66`	– (unregistered)	–

The rest of the system only ever sees "card *UID* scanned at side *IN/OUT*" (`components/nodes/include/rfid.h`), so any UID, including unregistered ones, can also be scanned from the CLI with `tap <UID> in|out`.

4.2 Pin map

`components/common/include/board_pins.h` is the single source of truth. Pins follow the board's physical header order, which keeps the diagram's wiring tidy (section 4.3).

Header	Function	GPIO	Notes
left	Room 1: LDR / PIR / lamp	4 / 5 / 6	LDR on ADC1_CH3

Header	Function	GPIO	Notes
left	Room 2: LDR / PIR / lamp	7 / 15 / 16	LDR on ADC1_CH6
left	SD card MISO / SCK / MOSI / CS	10 / 11 / 12 / 13	SPI2 (SD card only)
left	Buzzer	14	LEDC PWM
right	Entry reader SDA (chip select)	1	SPI3
right	RFID SCK / MOSI / MISO	2 / 42 / 41	SPI3, shared by both readers
right	RFID RST	40	both readers, active low
right	Exit reader SDA (chip select)	39	SPI3
right	I2C SDA / SCL (LCD, RTC)	36 / 35	one shared bus
right	Entry / exit door servo	48 / 47	LEDC 50 Hz
right	Break-glass button	21	pull-up, interrupt
right	"Access granted" LED	20	

Peripheral sharing. The SD card has SPI bus 2 to itself; the two readers share SPI bus 3 and are told apart by their chip-select pins, so a card read never waits for an SD write. The LCD and RTC share one I2C bus (different addresses). The ESP32 LED controller (LEDC) provides all PWM: timer 0 at 50 Hz for both servos, timer 1 at 5 kHz for the two lamps, and timer 2 for the buzzer, whose frequency changes per tone. The two light sensors share ADC unit 1, protected by a mutex (section 7).

4.3 Wiring

The Wokwi diagram is **generated**, not drawn by hand. `firmware/tools/gen_diagram.py` reads the GPIO numbers from `board_pins.h`, places each part beside the header pins it uses, routes every wire with explicit horizontal and vertical segments, adds a text label to every group, and then checks the result: no two different nets overlap and no wire runs through a part. Point-to-point signals never cross; the readers' shared bus lines and the power and ground rails cross other wires only at right angles. `make diagram` regenerates both `diagram.json` and fig. 3.

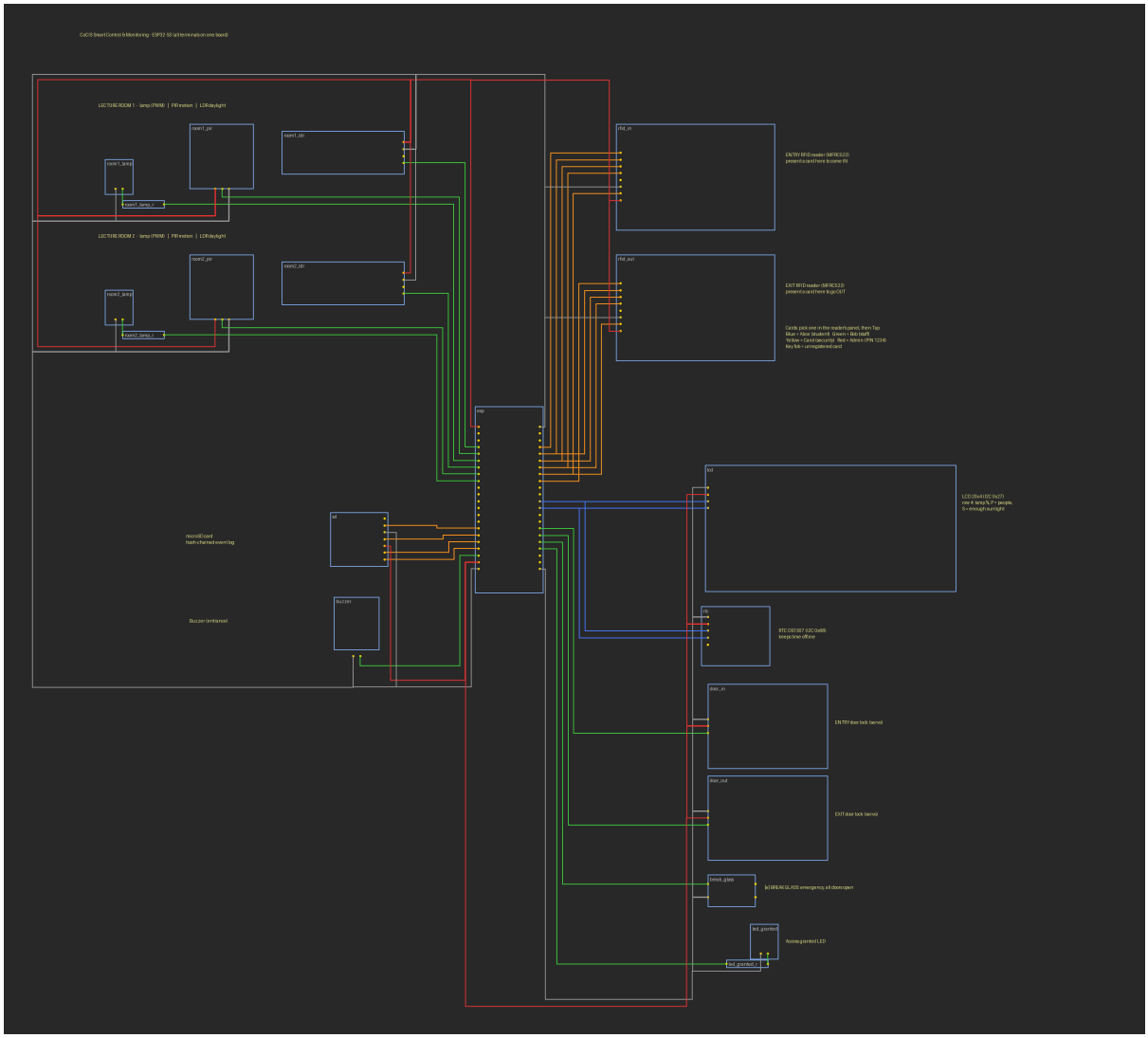


Figure 3: Wiring layout rendered by `gen_diagram.py -preview`. Left: the two lecture rooms, the SD card and the buzzer. Centre: the ESP32-S3. Right, top: the entry and exit RFID readers (orange: SPI). Right, below: LCD, RTC, the two door servos, break-glass button and granted LED. Red = power, grey = ground.

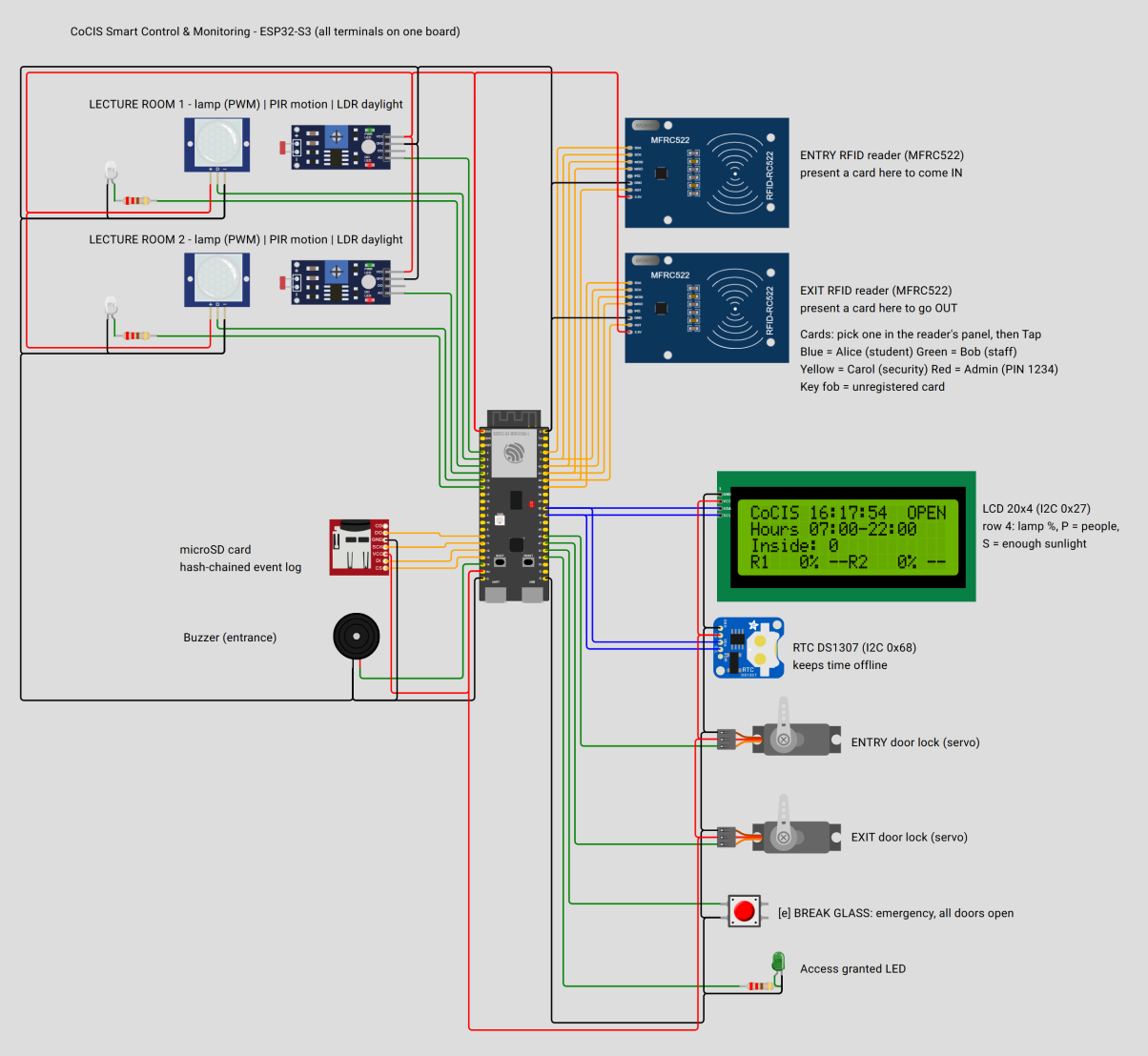


Figure 4: The complete circuit running in the Wokwi simulator.

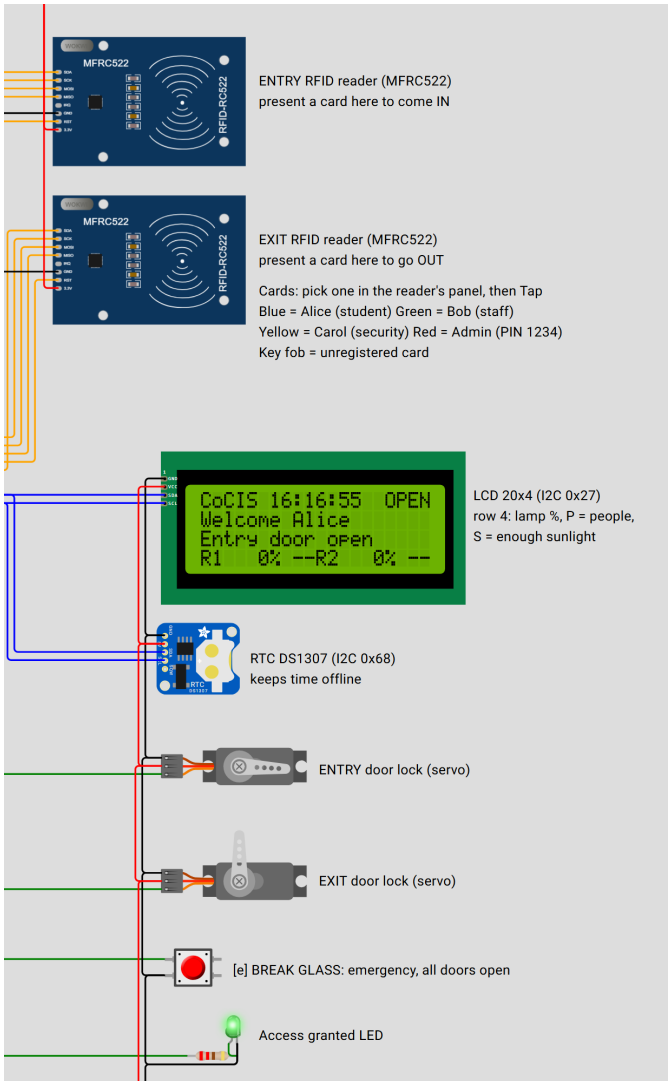


Figure 5: Entrance terminal: entry and exit RFID readers, LCD, RTC, door servos, break-glass button and granted LED.

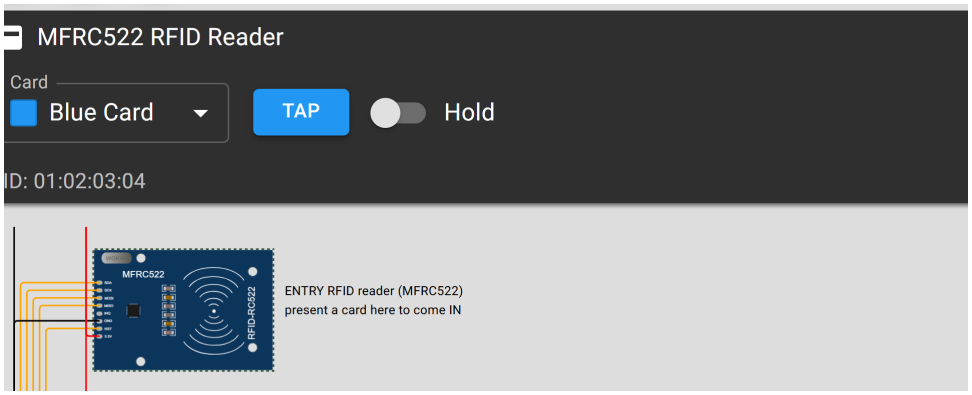


Figure 6: An RFID reader's control panel in Wokwi: pick a card, then Tap or Hold it.

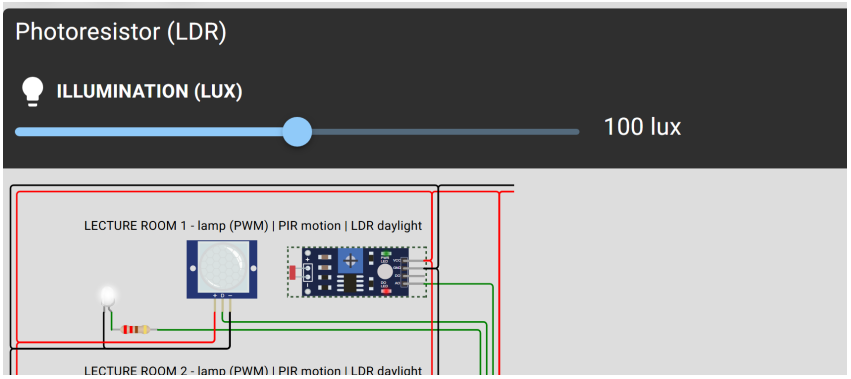


Figure 7: A lecture-room terminal: PIR sensor, light sensor and lamp. The PIR and light-sensor controls are opened by clicking the part.

5 Software Design

5.1 Toolchain and project layout

The firmware is an ESP-IDF v6.0.3 project for the ESP32-S3 (`firmware/`), written in C11. It builds with no warnings and uses about 26 % of the application partition. The same sources also build with the wokwi.com online builder (ESP-IDF v5.3), which is how the shared Wokwi link runs (section B.2).

```
firmware/
  main/              main.c (boot order), Kconfig.projbuild (tunable settings)
  components/
    common/          types, pin map, task table, SHA-256
    core_logic/      access_policy, room_logic, auth_fsm   <- pure, unit tested
    drivers/         i2c_bus, lcd2004, ds1307, mfrc522, buttons, pwm, actuators, light_sensor
    bus/             messages between terminals
    storage/         SD card gatekeeper + hash-chained log
    services/        users, config, occupancy, auth, clock (+ nvs_store helper)
    ui/              LCD, buzzer, LED task
    nodes/           hub, hub_access, room_node, door, rfid
    cli/             serial menu: cmd_monitor, cmd_admin, cmd_sim
  test/host/         PC unit tests for core_logic
  tools/             gen_diagram.py (generates diagram.json)
  diagram.json, wokwi.toml   Wokwi simulation
```

Every source file starts with a comment block that says what the module does and why, and every public function is documented with its parameters, return value and the tasks allowed to call it.

5.2 FreeRTOS tasks

Work is divided into tasks so that a slow job (an SD write, a buzzer tone) never delays a fast one (a door opening). Stack sizes and priorities are kept together in `components/common/include/task_config.h`; a higher number is a higher priority.

Task	Prio	Stack	Job
`rfid`	6	3 KB	Polls both RFID readers every 100 ms, reads a newly presented card's UID, sends `MSG_CARD_SCAN`; re-initialises a reader that stopped answering.
`buttons`	6	3 KB	Debounces the break-glass button (interrupt → queue → task), sends `MSG_BREAK_GLASS`.
`hub`	5	6 KB	Entrance terminal: access decisions, admin login, room supervision, energy log.
`room1`, `room2`	4	4 KB	Room terminals: sense every 200 ms, decide the lamp level, report to the hub.
`storage`	3	8 KB	SD card gatekeeper: appends, tails and verifies the log.
`ui`	2	4 KB	Redraws the LCD every 500 ms, plays sounds, times the LED.
`cli`	1	8 KB	Serial command menu (lowest priority: a human is typing).

The door auto-close uses one `esp_timer` per door (a callback, not a task).

Input tasks have the highest priority so no card or button press is missed. The storage, UI and CLI tasks are below the terminals because they only consume results.

5.3 Boot sequence

`main/main.c` brings the system up in dependency order, so each step can use the ones before:

1. NVS (flash storage), the I2C bus, and the clock (time from the DS1307 RTC, or the build time as a fallback).
2. The storage task (mounts the SD card and resumes the log chain), then a SHA-256 self-test. A failure is logged as `ALERT code=CRYPTO_FAIL`.
3. Shared state: users, configuration, occupancy ledger, auth.
4. The bus and the light sensor ADC.
5. The hub task, then room nodes 1 and 2.
6. A `BOOT` log line with the reset reason, then the serial CLI.

5.4 The node bus

`components/bus/` is how the terminals talk. Each node (`HUB`, `ROOM1`, `ROOM2`) has an inbox, a FreeRTOS queue of 16 messages. A message is a tagged union:

Type	Direction	Payload
`MSG_CARD_SCAN`	RFID reader → hub	card UID, side (IN/OUT)
`MSG_BREAK_GLASS`	button → hub	–
`MSG_ROOM_STATE`	room → hub	occupied, daylight, level, lux, mode, energy used/saved
`MSG_HEARTBEAT`	room → hub	uptime
`MSG_LIGHT_CMD`	hub → room	light mode (AUTO, ON, OFF)

`bus_send()` stamps each message with its source and the next **sequence number** for that sender. `bus_recv()` remembers the last sequence number seen from each sender and drops any message that is not newer, so a duplicated or replayed frame is never processed twice. The number dropped is shown by `status`.

5.5 RFID reader driver

`components/drivers/mfrc522.c` talks to the MFRC522 chip over SPI. The chip is a radio front end: the ESP32 puts bytes in its 64-byte FIFO, starts a *Transceive* command, and reads the card's answer back from the FIFO. Reading a card's UID follows ISO/IEC 14443-3 type A:

1. **REQA** (or **WUPA** for a card that was put to sleep): "is a card there?". A card answers with its 2-byte ATQA.
2. **ANTICOLL**: the card sends 4 UID bytes and their XOR check byte (BCC), which the driver verifies.
3. **SELECT** with a CRC_A: the card confirms and answers SAK, which says whether more UID bytes follow (7-byte UIDs take two such levels).
4. **HLTA**: the card goes to sleep, so a card left on the reader is not read again and again.

`components/nodes/rfid.c` owns SPI bus 3 and both readers. Its task polls each reader every 100 ms. A card counts as a *new* presentation when that reader has not seen it during the last 1.5 s, so a card held on the reader opens the door once, and presenting it again later is a new scan. A 4-byte UID is used as the 32-bit card number first byte first (Blue card `01:02:03:04` →

01020304); a 7-byte UID keeps its last 4 bytes. At boot the task resets both readers and checks each chip answers; a reader that does not is logged as `ALERT code=RFID_FAIL` and retried every 5 s, and `status` shows the readers' health.

5.6 Pure decision logic (`core_logic`)

The rules that matter most are plain C functions with no hardware access: they take a struct of facts and return a decision. This keeps the rules short, readable and testable on a PC (section 9). The access policy is the clearest example:

```
access_reason_t access_decide(const access_request_t *req)
{
    if (!req->known) return ACCESS_UNKNOWN_CARD;
    if (!req->enabled) return ACCESS_DISABLED;
    if ((req->side == SIDE_IN && req->inside) || (req->side == SIDE_OUT && !req->
        inside))
        return ACCESS_ANTI_PASSBACK;
    bool privileged = role_is_privileged(req->role); /* SECURITY or ADMIN */
    if (req->lockdown) return privileged ? ACCESS_OK_PRIVILEGED : ACCESS_LOCKDOWN;
    if (req->schedule_open) return ACCESS_OK;
    if (privileged) return ACCESS_OK_PRIVILEGED;
    if (req->side == SIDE_OUT && req->escort_active) return ACCESS_OK_ESCORT;
    return ACCESS_OUTSIDE_HOURS;
}
```

(Shown compacted; the real function in `access_policy.c` has the same logic.) `room_logic_step()` does the same for lighting (section 6.3) and `auth_fsm` for the admin login (section 6.5).

5.7 Shared state services

Each module in `components/services/` owns one piece of shared state, guards it with its own mutex, hands out copies to readers, and saves it to NVS (the ESP32's key-value flash store) so it survives a reboot:

- `users`: the card registry (UID, name, role, enabled, salted SHA-256 PIN hash). It is versioned, so a changed layout re-seeds the four demo cards.
- `config`: the access schedule and lockdown flag. A change is saved to NVS *before* it becomes live, so a power cut never leaves the two disagreeing.
- `occupancy`: the ledger of who is inside, saved on every change.
- `auth`: wraps the pure `auth_fsm` and writes every login result to the log.
- `clock`: sets the system time from the DS1307 at boot (time zone EAT, UTC+3).

5.8 Interrupts

Interrupt handlers do the minimum: they push the pin number to a queue or notify a task, and return. Debouncing, scanning and decisions happen in tasks. This keeps interrupts short and makes it safe to call any FreeRTOS or logging function from the code that handles the press.

5.9 Coding principles

- **Single source of truth.** Pins are only defined in `board_pins.h`, task settings only in `task_config.h`, tunable behaviour only in `Kconfig.projbuild`, and each enum and its string names only once, using X-macros in `cocis_types.h`.
- **One way to do each thing.** All modules log through `cocis_log(type, "k=v ...")`. One PWM layer drives servos, lamps and the buzzer. One button driver (interrupt, queue, debounce) serves any push button. Every card scan, from either reader or the CLI's `tap`, goes through the same `rfid_inject()`. One NVS blob helper is used by all persistent modules.
- **Separation of concerns.** Drivers know hardware, services know state, nodes combine them, and `core_logic` knows only the rules.

6 System Operation and Implemented Features

This section explains how each feature works, step by step, and names the code that does it.

6.1 Access control at the entrance (R2.1, R2.6)

Users and roles. Each card has a UID, a name, a role and an enabled flag. The four Wokwi preset cards are registered on first boot (presented at a reader as described in section 4.1):

Wokwi card	UID	Name	Role	PIN
Blue	`01020304`	Alice	STUDENT	–
Green	`11223344`	Bob	STAFF	–
Yellow	`55667788`	Carol	SECURITY	–
Red	`AABBCCDD`	Admin	ADMIN	`1234`

SECURITY and ADMIN are *privileged* roles: they may pass at any time. Cards can be added, removed, enabled and disabled from the CLI by an admin.

What happens when a card is scanned. `components/nodes/hub_access.c`, `hub_access_handle_scan()`:

1. The rfid task reads the card at the entry or exit reader and sends `MSG_CARD_SCAN` (UID + that reader's side) to the hub. If an admin login is waiting for its card and this is an admin card, the scan is the login's first factor instead (section 6.5).
2. The hub collects the facts: is the card known and enabled, what role, is the person inside now (ledger), is the building open now (schedule + clock), is lockdown on, is an escort window open.
3. `access_decide()` returns the verdict, using the rules below, first match wins.
4. If granted, the ledger is updated (in or out) and *only the door on that side* opens: its servo turns to 90° and an `esp_timer` closes it after 3 s.
5. An `ACCESS` line is logged with the UID, name, role, side, result and reason.
6. The LCD shows "Welcome Alice / Entry door open" or "DENIED Alice / ANTI_PASSBACK", with a short beep and the green LED, or three low beeps.

#	Rule	Result
1	Card unknown or disabled	`UNKNOWN_CARD` / `DISABLED`
2	**Anti-passback**: entering while already inside, or leaving while outside	`ANTI_PASSBACK`. This stops a card being passed back to a friend and keeps the "who is inside" list truthful.
3	Lockdown is on	Only privileged roles pass (`OK_PRIVILEGED`), others get `LOCKDOWN`
4	Inside the opening hours	`OK`
5	Outside hours, privileged role	`OK_PRIVILEGED`
6	Outside hours, ordinary user leaving while an escort window is open	`OK_ESCORT`
7	Otherwise	`OUTSIDE_HOURS`

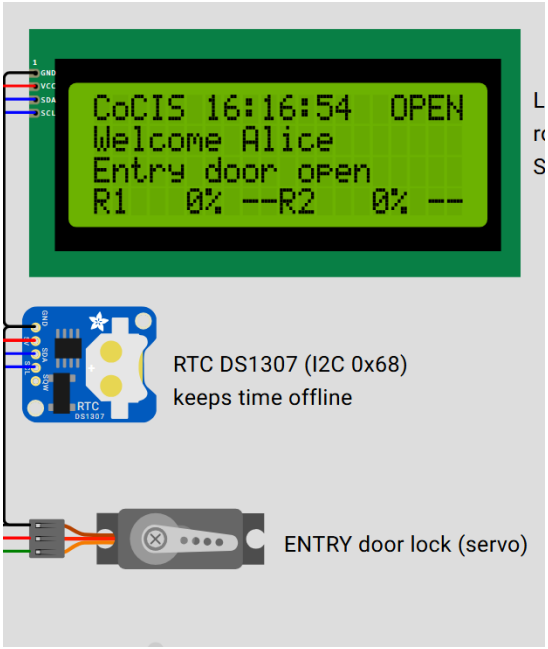


Figure 8: LCD after a granted entry scan.

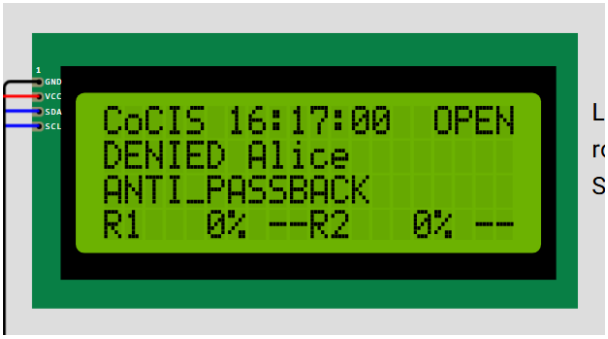


Figure 9: LCD after a refused scan (anti-passback).

```
[LOG] 000008|2026-10-28T11:35:49|ACCESS|uid=01020304 name=Alice role=STUDENT side=IN ok=1
    reason=OK
[LOG] 000009|2026-10-28T11:35:51|ACCESS|uid=01020304 name=Alice role=STUDENT side=IN ok=0
    reason=ANTI_PASSBACK
[LOG] 000010|2026-10-28T11:35:53|ACCESS|uid=01020304 name=Alice role=STUDENT side=OUT ok=1
    reason=OK
[LOG] 000011|2026-10-28T11:35:54|ACCESS|uid=C0FFEE99 name=unknown role=NONE side=IN ok=0 reason
    =UNKNOWN_CARD
```

Figure 10: Log records of four scans at the real readers: Alice enters, tries to enter again (anti-passback), leaves at the exit reader, then an unregistered key fob (captured in Wokwi).

Leaving after hours. The brief requires that people still inside after the access period are not allowed to leave (**R2.6**). We enforce this, and add two safety measures:

- When an ordinary user is refused at the exit after hours, the hub logs `ALERT code=AFTER_HOURS_EXIT`, so security knows someone is waiting.
- When a **security** card exits after hours, an **escort window** opens for 60 s, during which

ordinary users may leave (`OK_ESCORT`). Security thus walks people out and every exit is still recorded.

- The **break-glass** button overrides every rule for life safety (section 6.9).

6.2 Occupancy records (R2.1)

The occupancy ledger (`services/occupancy.c`) is the list of cards currently inside. Only the hub changes it, after a granted scan, and it is saved to NVS on every change, so a reboot does not forget who is inside (the boot log prints "2 people inside at boot"). `who` prints the list with entry times. This is the muster list for an evacuation. Together with the `ACCESS` lines on the SD card, management can see who is inside now and who came and went at any time.

```
cocis> who
3 people inside the building:
  01020304  Alice           STUDENT   since 2026-10-28 11:42
  11223344  Bob             STAFF     since 2026-10-28 11:42
  55667788  Carol           SECURITY  since 2026-10-28 11:42
```

Figure 11: `who` after Alice, Bob and Carol entered (captured in Wokwi).

6.3 Automatic lighting in the lecture rooms (R2.2)

Each room task (`nodes/room_node.c`) runs this loop every 200 ms:

1. Read any command from the hub (light mode AUTO, ON or OFF).
2. Read the PIR (motion yes/no) and the LDR (light level in lux).
3. Call the pure `room_logic_step()`, which decides occupancy, daylight and the lamp level.
4. Move the lamp towards that level by 25 % per step, a soft 0.8 s fade done in software.
5. Report the new state to the hub when something changed (and every 60 s), and send a heartbeat every 5 s.

The lighting rules (`core_logic/room_logic.c`, AUTO mode):

- **Occupancy.** Motion makes the room occupied. With no motion for the *grace period* (30 s) the room becomes vacant.
- **Pre-off warning.** During the last 10 s of the grace period the lamp dims to 30 %. Anyone still in the room moves and the light comes back to 100 %, instead of being plunged into darkness.
- **Daylight harvesting.** At or above 400 lux the room counts as bright and the lamp stays off even if occupied. At or below 150 lux it is dark again. Between the two, the last decision is kept (*hysteresis*), so a cloud passing near one threshold does not make the light flicker.
- **Lamp level** is therefore 100 % if occupied and dark, 30 % during the warning, and 0 % if vacant or bright. ON and OFF (set by an admin with `light`) override it.

Energy accounting. Each step adds the lamp's energy at its current level (360 W lamp bank per room, configurable) to *energy used*. During opening hours it also adds full power to a *baseline*: what a room whose lights are simply left on all day would use. *Energy saved* = baseline − used.

The hub logs `ENERGY room=1 on_wh=... saved_wh=...` for each room every 10 minutes, and `rooms` shows the totals. This gives management a figure for the electricity the system saves.

```
cocis> rooms
Room  Link     Occupancy  Light  Mode  Lux    Used Wh  Saved Wh
1     online   vacant         0%  AUTO  650          0.0       0.4
2     online   vacant         0%  AUTO  100          0.0       0.0
```

Figure 12: `rooms` with daylight in Room 1 (650 lux): vacant, lamp off, energy saved so far (captured in Wokwi).

6.4 Configurable access schedule (R2.6)

The schedule is an opening time, a closing time and the weekdays it applies to. The default is 07:00–22:00, Monday to Saturday. An admin changes it at any time:

```
cocis> schedule set 07:00 18:00 weekdays
```

If the closing time is earlier than the opening time, the window runs overnight (e.g. 22:00–06:00, where the early-morning part belongs to the previous day's window). The new schedule is validated, saved to NVS, takes effect immediately, and is logged as `CONFIG`. The LCD shows the hours on row 2 and OPEN or SHUT on row 1. Changing the schedule requires an admin session, so only authenticated personnel can do it (section 6.5).

```
cocis> schedule set 07:00 18:00 weekdays
Admin session required. Run 'login', tap the admin card, then 'login <PIN>'.
cocis> login
Tap the ADMIN card (Red card) at either RFID reader within 30 s,
then type 'login <PIN>' here.
cocis> [LOG] 000012|2026-10-28T11:42:46|AUTH|uid=AABBCCDD result=CARD_OK
cocis> login 1234
Logged in: admin session started.
cocis> schedule set 07:00 18:00 weekdays
Schedule updated.
cocis> [LOG] 000014|2026-10-28T11:42:46|CONFIG|key=schedule value=07:00-18:00_-MTWTF- version=2
     by=AABBCCDD
```

Figure 13: Changing the schedule: refused without a session; `login`, the Red admin card at the exit reader, the PIN; then accepted and logged as `CONFIG` (captured in Wokwi).

6.5 Two-factor admin authentication (R2.6)

Configuration commands need an admin session, which takes **two factors**: something you have (the admin card) and something you know (a 4-digit PIN). The pure state machine `core_logic/auth_fsm.c` drives it:

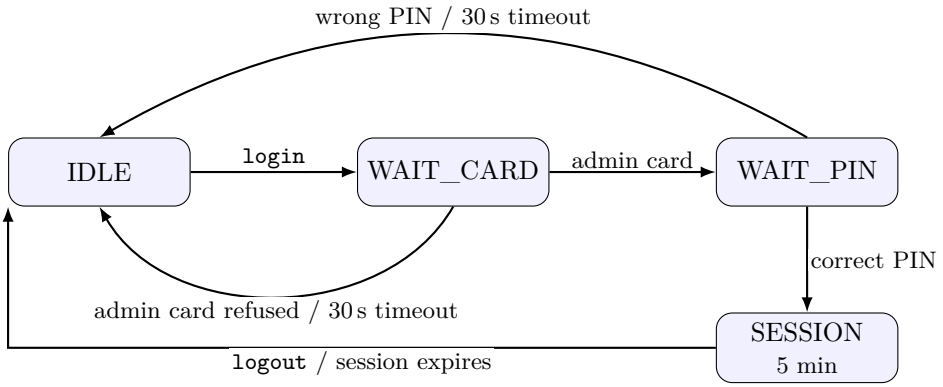


Figure 14: Admin login state machine. Three wrong PINs lock login for 60 s.

1. Type `login` on the serial console.
2. Within 30 s, present the admin card (Red) at either reader. The LCD shows "Admin login / PIN on console". Any other card presented meanwhile is handled as a normal door request, so people keep coming and going during the login.
3. Type `login 1234`. PINs are exactly 4 digits.

PINs are never stored in clear text: the registry stores SHA-256 of a per-user random salt plus the PIN. Every step (`CARD_OK`, `OK`, `BAD_PIN`, `LOCKED`, `TIMEOUT`, `LOGOUT`) is logged as an `AUTH` line, and a lockout also raises `ALERT code=PIN_LOCKOUT`. The session ends after 5 minutes or on logout.

The console session in fig. 13 shows the whole login.

6.6 Data storage on the SD card (R2.4)

Everything that happens becomes one text line on the SD card, in a new file every day: `/sdcard/LOGS/YYYYMMDD.LOG`.

```
000010|2026-10-28T09:46:10|ACCESS|uid=01020304 name=Alice role=STUDENT side=IN ok=1 reason=OK|3
    f9a...c2
000014|2026-10-28T09:47:04|ROOM|room=1 occupied=0 daylight=0 lux=125|81d0...7e
```

The fields are: a sequence number, the local time, an event type, `key=value` details, and a hash. The event types are `BOOT`, `ACCESS`, `ROOM`, `LIGHT`, `ENERGY`, `CONFIG`, `USER`, `AUTH` and `ALERT`; the full list of fields is in `docs/event-schema.md`.

Tamper-evident hash chain. The last field of each line is $h_n = \text{SHA-256}(h_{n-1} \,\|\, \texttt{"|"} \,\|\, \text{line}_n)$, with $h_0 = 64$ zeros at the start of each file. Changing, inserting or deleting any line changes every hash after it. `log verify` recomputes the chain and reports the first broken line, so management can trust that the records were not edited.

Reliability. Only the storage task writes to the card. Each line is flushed and `fsync`ed, so a power cut loses at most the line being written. After a reboot, the task reads the last line to continue the sequence number and the chain. If the card is missing, logging continues in a RAM ring buffer of the newest 32 lines and `ALERT code=SD_FAIL` is raised. `log tail` shows recent lines either way.

```
cocis> log tail 8
000008|2026-10-28T11:42:42|ACCESS|uid=01020304 name=Alice role=STUDENT side=IN ok=1 reason=OK
000009|2026-10-28T11:42:43|ACCESS|uid=11223344 name=Bob role=STAFF side=IN ok=1 reason=OK
000010|2026-10-28T11:42:44|ACCESS|uid=55667788 name=Carol role=SECURITY side=IN ok=1 reason=OK
000011|2026-10-28T11:42:45|ROOM|room=1 occupied=0 daylight=1 lux=650
000012|2026-10-28T11:42:46|AUTH|uid=AABBCCDD result=CARD_OK
000013|2026-10-28T11:42:46|AUTH|uid=AABBCCDD result=OK
000014|2026-10-28T11:42:46|CONFIG|key=schedule value=07:00-18:00_-MTWTF- version=2 by=AABBCCDD
000015|2026-10-28T11:42:46|CONFIG|key=schedule value=07:00-22:00_-MTWTFS version=3 by=AABBCCDD
cocis> log verify
OK: 15 lines, hash chain intact
```

Figure 15: `log tail` and `log verify`: the newest records and the hash-chain check (captured in Wokwi).

6.7 Serial interface and management menus (R2.5)

The serial console (`components/cli/`) gives a `cocis>` prompt with history and `TAB` completion. `help` lists all commands; section A gives the full reference.

Group	Commands
Monitoring (anyone)	`status` overview: time, schedule, open/closed, people inside, rooms, storage, bus statistics; `who` muster list; `rooms` occupancy, lamp level, lux, energy; `users` registered cards; `log tail [n]`, `log verify [date]`, `log echo on|off`
Configuration (admin)	`schedule set`, `user add|del|enable|disable`, `light <room> auto|on|off`, `lockdown on|off`, `emergency clear`, `time set`
Session	`login`, `logout`
Simulation	`tap <UID> in|out` (any card at a reader), `node <1|2> offline|online` (simulate a room losing its link)

With `log echo on`, every new log line is also printed live on the console as `[LOG] ...`, which turns the terminal into a live event monitor.

```
cocis> status
Time         : 2026-10-28 11:42:46 Wed (EAT)
Uptime       : 5 s, free heap 276400 B
Schedule     : 07:00-22:00_-MTWTFS (v3)  -> OPEN, closes in 618 min
Lockdown     : off   Emergency: off   Escort window: closed
Inside       : 3 people
Access       : 3 granted, 0 denied, 2 alerts
Storage      : SD card OK, last seq 15
Bus          : 0 duplicate frames rejected
RFID readers: entry OK, exit OK
Admin        : logged in as AABBCCDD (299 s left)
Room 1       : online, vacant, light   0% (AUTO), 650 lux
Room 2       : online, vacant, light   0% (AUTO), 100 lux
```

Figure 16: `status`: the system overview (captured in Wokwi).

6.8 LCD, buzzer and LED

A single `ui` task owns these outputs. Other tasks send it requests through a queue and never wait for it. The 20×4 LCD shows:

```
CoCIS 09:15:45  OPEN      <- clock and building state: OPEN, SHUT, LOCK or EMRG
Hours 07:00-22:00         <- or a 2-line message, e.g. "Welcome Alice / Entry door open"
Inside: 3       ADMIN     <- people in the building, admin session active
R1 100% P-R2    0% -S     <- per room: lamp %, P = people detected, S = enough sunlight
```

Only rows whose text changed are rewritten, so there is no flicker. The buzzer has distinct patterns for granted, denied, information and alarm.

6.9 Emergency, lockdown and node supervision

- **Break-glass** (key `e`): both doors are held open, the alarm sounds, the LCD shows EMRG and `ALERT code=BREAK_GLASS` is logged. Only an admin can end it, with `emergency clear`.
- **Lockdown** (`lockdown on`): only security and admin cards pass, for example during an incident.
- **Node supervision.** Rooms send a heartbeat every 5 s. If the hub hears nothing for 15 s it marks the room offline, shows `R2 offline` on the LCD and logs `ALERT code=NODE_OFFLINE`, and `NODE_ONLINE` when it returns. The room keeps controlling its own lights meanwhile.

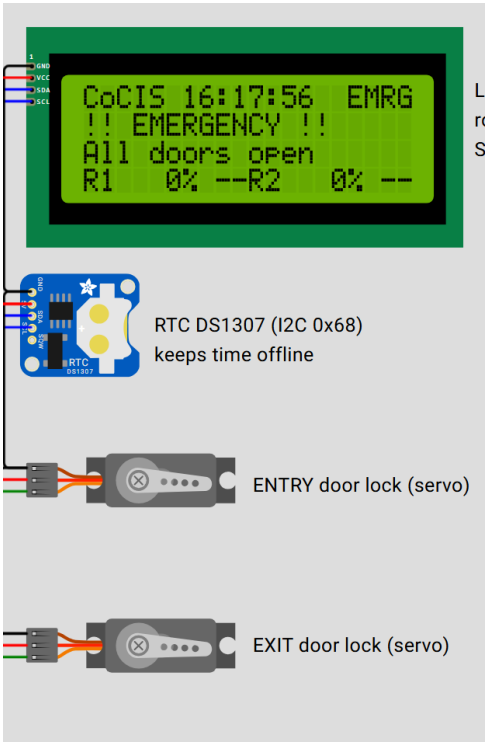


Figure 17: Break-glass emergency: both doors held open, EMRG on the LCD.

7 Synchronisation and Reliability

Several tasks run at the same time and some resources are shared, so **R2.7** is met with a small set of rules, each tied to a specific risk.

7.1 Design rules

1. **One writer per record.** Only the hub task decides access and changes the occupancy ledger; card scans from any source are *queued* to it and processed one at a time. Two people scanning at once can never produce an inconsistent ledger.
2. **One owner per device.** Only the storage task touches the SD card, and only the ui task touches the LCD. Other tasks send requests through queues.
3. **One mutex per shared module.** Users, config, occupancy, auth, the light-sensor ADC and the hub status each have their own lock, held only for a quick copy. Readers (CLI, LCD) get a copy, never a pointer into live data.
4. **Interrupts never block.** They only queue an event or notify a task.

7.2 Shared resources and how each is protected

Resource	Risk without protection	Mechanism
Access decisions and ledger	Two scans decided at once give a wrong "who is inside"	Single writer: the hub task, fed by its inbox queue.
SD card (SPI + FAT)	Interleaved writes corrupt files or mix lines	Gatekeeper task. Log requests are queued; `log tail` and `log verify` are requests answered through a semaphore.
Log sequence numbers	Gaps or duplicates	Assigned only by the gatekeeper; resumed from the last line on the card after a reboot.
Bus sequence numbers	Two tasks sending as the same node (e.g. the rfid task and the CLI's `tap`) could enqueue seq 11 before seq 10, which the receiver would drop as stale	A per-sender mutex makes "take next seq + enqueue" one step. *We found this race in code review and fixed it.*
Duplicated or replayed messages	Processed twice	The receiver drops any seq not newer than the last one from that sender.
Users, config, occupancy, auth	A reader sees a half-finished update	One mutex per module; copies out.
Config in flash vs. in RAM	Power cut after the change but before the save	Saved to NVS first (atomic commit), then made live.
RFID readers on one SPI bus	Two transfers mixed on the shared wires	One owner: only the rfid task uses SPI bus 3, one reader at a time (each has its own chip-select). The SD card has its own bus.
Card held on a reader	One person counted many times	A card is a new scan only after 1.5 s unseen at that reader, and the card is halted after each read.
ADC unit shared by both rooms	Interleaved conversions give wrong readings	Mutex in `light_sensor.c`.
I2C bus (LCD + RTC)	Mixed transactions	ESP-IDF's I2C master bus lock per transaction.

Resource	Risk without protection	Mechanism
Door close timer vs. a new open	Door closes on the next person	The timer restarts on every open.
Admin login	A PIN typed late, or a card scan and the CLI acting at once	One state machine behind a mutex; deadlines are checked with wrap-safe time arithmetic.
Power loss	Half-written state	Atomic NVS commits; each log line `fsync`ed; ledger saved on every change.
Room node failure	Lights uncontrolled without anyone knowing	Heartbeats and an offline alert; rooms run independently of the hub.
SD card missing	All records lost	RAM ring buffer and `ALERT code=SD_FAIL`.

7.3 Coordination between terminals

The terminals coordinate only through bus messages (section 5.4). A room never waits for the hub: if a send fails, the room keeps its lights correct and tries again with the next report. The hub never waits for a room either; it uses the latest state each room reported and marks a room offline when its heartbeats stop. This is the same failure model that three separate boards would have.

8 Innovation and Additional Features

The brief places particular emphasis on innovation. Beyond the core requirements, the system adds the following, grouped by the goal named in the brief.

8.1 Security

- **Tamper-evident log.** Every SD line is chained with SHA-256; `log verify` proves that no record was edited, inserted or deleted (section 6.6).
- **Two-factor admin login** (card + PIN) with salted PIN hashes, 3-attempt lockout, 30 s step timeouts and a 5-minute session (section 6.5).
- **Real RFID readers at both doors** (MFRC522, ISO 14443 A), with a driver written from the datasheet: UID check bytes verified, each presentation counted once, and a reader health check (`ALERT RFID_FAIL`, automatic retry).
- **Anti-passback**, so one card cannot be used by two people.
- **Lockdown mode**: only security and admin may pass.
- **Replay protection** on the bus: duplicated or replayed messages are dropped.
- **Boot self-test** of SHA-256 against the standard test vector.

8.2 Efficiency

- **Daylight harvesting with hysteresis**: lights stay off in a bright room even when it is occupied.
- **Energy accounting**: energy used and saved per room, logged every 10 minutes, so management can see what the system saves.
- **Pre-off dim warning** and soft fades: lights dim before turning off, so occupants are never left in the dark.

8.3 Reliability and safety

- **Break-glass override** and the **security escort window**: after-hours rules never trap anyone in an emergency, and every exit is still recorded.
- **After-hours exit alert**: security is told when someone is waiting to leave.
- **Node heartbeats** with offline and online alerts.
- **Graceful degradation**: no SD card → RAM log; no RTC → build time; hub down → rooms keep working.
- **Persistence**: users, schedule, lockdown and the occupancy ledger survive reboots.

8.4 Usefulness to management

- A muster list (`who`) with entry times for evacuations.
- A live event monitor on the console (`log echo on`).
- A clear LCD summary: building state, people inside and every room's status at a glance.

8.5 Engineering quality

- **Unit-tested decision logic** on a PC (70 checks), so the rules are proven before they run on the board (section 9).
- **Generated, self-checking wiring diagram** from the pin map, so the circuit and the code can never disagree (section 4).
- **Tunable without code changes**: all timings and thresholds are `menuconfig` options.

8.6 Optional cloud dashboard (prepared extension)

`dashboard/` contains a Next.js 16 web application for Vercel, storing data in a Neon Postgres database through Prisma 7 (versioned SQL migrations, a node-postgres driver adapter on Neon's pooled endpoint). It receives the same events as the SD log through `POST /api/ingest`:

- Requests are authenticated with an HMAC-SHA256 signature and a timestamp; stale or forged requests are rejected.
- Ingest is idempotent: each (device, sequence number) pair is stored once, so a re-sent batch creates no duplicates.
- Pages show the people inside, room status, alerts, a filterable access log and energy saved.

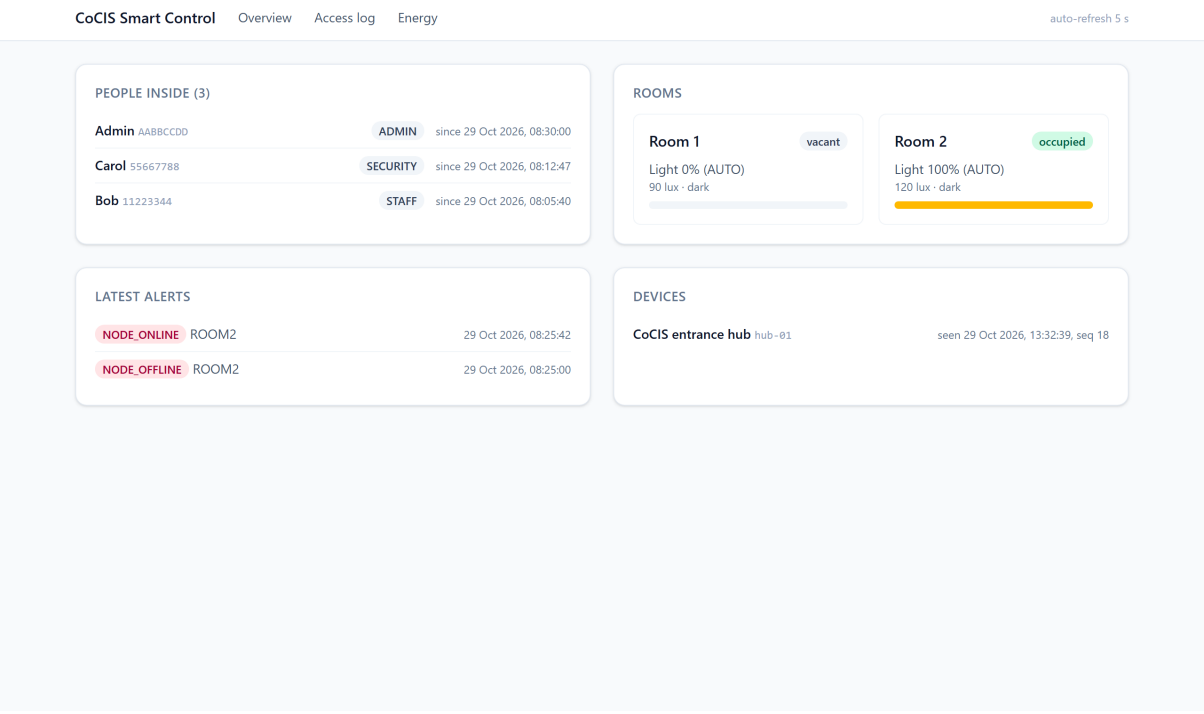


Figure 18: Dashboard overview page with sample events: people inside, rooms and alerts.

It was tested end to end with a script that sends signed sample events. The firmware does *not* yet send events to it (section 10), and the system never depends on it.

9 Testing

9.1 Unit tests of the decision logic

`firmware/test/host/test_core_logic.c` compiles the pure `core_logic` and the type helpers with the PC's gcc (or a gcc Docker image) using `-Wall -Wextra -Werror`, and checks them against expected results. Run it with `make test`.

Group	What it proves
Schedule	Opening and closing edges, closed weekdays, overnight windows, minutes to close, invalid schedules rejected.
Access policy	Every rule in order: unknown, disabled, anti-passback both ways, lockdown, in hours, privileged after hours, escort exit, after-hours refusal.
Room logic	Occupancy and grace period, dim warning, daylight thresholds and hysteresis, manual ON/OFF override, energy used/saved, the daylight change flag.
Admin login	Happy path, non-admin card, wrong PIN, lockout after 3 failures and its expiry, step timeouts, late answers, session expiry, millisecond-counter wrap-around.
Types	Parsing of role names and card UIDs, including invalid input.

Result: 70 checks, 0 failures.

9.2 Build

`idf.py build` for the ESP32-S3 with ESP-IDF v6.0.3 completes with **0 warnings**. The application image is 0x62A10 bytes (about 395 KB), leaving 74 % of the application partition free.

9.3 Simulation testing in Wokwi

The system is tested in the Wokwi simulator in two ways: interactively in VS Code, and automatically with the Wokwi CLI, which runs the real firmware image against `firmware/diagram.json` and follows a YAML scenario. A scenario presents the reader's preset cards, sets the light sensors, types console commands, and fails unless each expected line appears on the serial port:

```
wokwi-cli firmware --scenario test/wokwi/access.test.yaml --timeout 60000
wokwi-cli firmware --scenario test/wokwi/demo.test.yaml   --timeout 90000
wokwi-cli firmware --scenario test/wokwi/rules.test.yaml  --timeout 120000
```

`make sim` runs all three. All three pass. The online project (`https://wokwi.com/projects/479125502548914177`) was also built by wokwi.com's ESP-IDF v5.3 builder and run: it boots, logs card scans at the entry reader, and reports daylight changes, as the local build does. The console figures in section 6 are their captured output. Observed results:

Scenario	Result
Boot	RTC time read, SD card mounted, SHA-256 self-test passed, 4 demo cards seeded, both RFID readers ready (MFRC522 version 0x92), both rooms online (fig. 19).
Card scans	Blue, Green and Yellow cards at the entry reader: `ok=1`; the Blue card again: `ANTI_PASSBACK`; the Blue card at the exit reader: `OK`, side `OUT`; the key fob: `UNKNOWN_CARD` (fig. 10).
Reboot	The log resumed after the last sequence number; the ledger reported "2 people inside at boot".
Admin login	`login`, the Red card at a reader, `login 1234`: `AUTH result=CARD_OK`, then `result=OK`; the schedule change is refused before and accepted after (fig. 13).
Records	`who`, `rooms`, `status` and `log tail` match the scans; `log verify` reports the hash chain intact.
After hours	With the building closed, Bob is refused at the exit (`OUTSIDE_HOURS`) and `ALERT AFTER_HOURS_EXIT` is raised; Carol (security) leaves, which opens the 60 s escort window; Bob then leaves (`OK_ESCORT`) (fig. 20).
Lockdown	A student is refused with `LOCKDOWN`.
Node failure	`node 2 offline`: `ALERT NODE_OFFLINE` after the 15 s timeout; `NODE_ONLINE` when the link returns.
Break-glass	The button raises `ALERT BREAK_GLASS` and holds both doors open; `emergency clear` by the admin ends it and is logged.
Light sensor	Moving the slider below 150 lux logged `ROOM ... daylight=0`; above 400 lux `daylight=1`.
Energy	`ENERGY` lines appeared every 10 minutes for both rooms.

```
I (131) clock: Time from RTC: 2026-10-28 11:34
I (254) storage: SD card mounted at /sdcard
I (259) crypto: SHA-256 self-test passed
I (260) users: First boot: seeded 4 demo cards
I (297) occupancy: 0 people inside at boot
I (474) rfid: IN reader ready (MFRC522 version 0x92)
I (527) rfid: OUT reader ready (MFRC522 version 0x92)
[LOG] 000001|2026-10-28T11:34:46|ALERT|code=NODE_ONLINE detail=ROOM1
[LOG] 000004|2026-10-28T11:34:46|ALERT|code=NODE_ONLINE detail=ROOM2
[LOG] 000007|2026-10-28T11:34:46|BOOT|reason=POWER_ON
```

Figure 19: Boot log in Wokwi: storage, self-test, users, ledger and both RFID readers come up before the terminals start.

```
cocis> schedule set 00:00 00:01 all
000012|...|CONFIG|key=schedule value=00:00-00:01_SMTWTFS version=2 by=AABBCCDD
000013|...|ACCESS|uid=11223344 name=Bob role=STAFF side=OUT ok=0 reason=OUTSIDE_HOURS
000014|...|ALERT|code=AFTER_HOURS_EXIT detail=Bob
000015|...|ACCESS|uid=55667788 name=Carol role=SECURITY side=OUT ok=1 reason=OK_PRIVILEGED
000016|...|CONFIG|key=escort value=60s by=55667788
000017|...|ACCESS|uid=11223344 name=Bob role=STAFF side=OUT ok=1 reason=OK_ESCORT
cocis> schedule set 07:00 22:00 mon-sat
cocis> lockdown on
000019|...|CONFIG|key=lockdown value=1 by=AABBCCDD
000020|...|ACCESS|uid=01020304 name=Alice role=STUDENT side=IN ok=0 reason=LOCKDOWN
cocis> lockdown off
cocis> node 2 offline
000022|...|ALERT|code=NODE_OFFLINE detail=ROOM2
cocis> node 2 online
<break-glass button pressed>
000026|...|ALERT|code=BREAK_GLASS detail=doors_held_open
cocis> emergency clear
000027|...|ALERT|code=BREAK_GLASS detail=cleared_by_AABBCCDD
cocis> log verify
OK: 27 lines, hash chain intact
```

Figure 20: The rules scenario in Wokwi: after-hours exit refused and escorted, lockdown, a room terminal going offline, and break-glass (log lines shortened: `|...|` is the time stamp; console prompts shown where a command was typed).

9.4 Defects found and fixed

Testing in the simulator and code review found real bugs, which were fixed:

- **Crash loop when a lamp faded on.** The ESP32's hardware LED-fade interrupt fired endlessly in Wokwi and tripped the interrupt watchdog. Because a software reset does not clear that peripheral, every reboot crashed again. The backtrace pointed to `ledc_fade_isr`; we removed the hardware fade and fade the lamp in software from the room task instead.
- **Bus ordering race** (section 7): fixed with a per-sender mutex.
- **PIN format**: PINs are exactly 4 digits, checked when a PIN is set and when it is typed.
- **LCD overflow**: every row is now formatted to exactly 20 characters.
- **Late PIN after a timeout** was accepted by mistake; the state machine now checks expiry first, and a unit test covers it.

9.5 Dashboard

`prisma validate`, the TypeScript check and `next build` pass. The initial migration was applied to the Neon database with `prisma migrate deploy`, and replaying the migrations into a throwaway Docker Postgres matches the schema exactly. With the production build running against Neon, the sample sender inserted 18 events, then 18 duplicates on a second run (idempotent); a bad signature and a stale timestamp were rejected with HTTP 401, and a signed but malformed batch with HTTP 400. The overview, access-log and energy pages render the stored data (fig. 18).

10 Limitations and Future Work

10.1 Known limitations

- **One board simulates three terminals** (section 3.2). The terminals are isolated and talk only through the bus, but a separate-board transport (ESP-NOW or Wi-Fi) has not been implemented.
- **Card numbers are 32-bit.** 4-byte UIDs are used whole; 7-byte UIDs keep their last 4 bytes, so two 7-byte cards differing only in their first bytes would share a number.
- **The readers are polled** every 100 ms; their IRQ pin is not used.
- **The device does not send events to the dashboard yet.** The dashboard and its ingest API are tested on their own, with sample events.
- **Truncation of the newest log lines** cannot be detected by the hash chain on its own; an off-device copy (the dashboard) would cover that.
- **A few edge cases are not yet handled**, found while reviewing the code:
  - typing `login` during an admin session silently starts a new login;
  - a full occupancy ledger (64 people) does not record the next entry.

10.2 Future work

- Split the terminals onto three ESP32 boards using ESP-NOW for the same bus messages.
- A device-side sync task: Wi-Fi, a signed batch upload, and an upload cursor kept on the SD card so nothing is lost while offline.
- A local web page served by the hub over Wi-Fi.
- A closing-time warning on the LCD and buzzer 15 minutes before closing (the function `schedule_minutes_to_close()` is already written and tested).
- Use the readers' IRQ pin to wake the rfid task instead of polling.

11 Group Members, Contributions and Videos

11.1 Group member details

#	Full name	Student Number	Registration Number
1	MUJUNI INNOCENT	2400707155	24/U/07155/PS
2	MARTHA NAMUYIMBWA	2400709436	24/U/09436/PS
3	OCHEN FELIX	2500726621	25/U/26621
4	NAKIMULI JOLLYNE FLAVIA	2400700953	24/U/0953
5	TAMARA ALICE NAZZIWA	2500703586	25/U/03596/PS

Wokwi simulation: `https://wokwi.com/projects/479125502548914177`

Open the link and press play: Wokwi builds and runs the firmware (section B.2).

11.2 Individual contributions

The work was divided into five areas so that each member owns a clear slice of the system and can explain its code. All members took part in the design, testing, this report and the presentation.

Member	Area and responsibilities	Main files
MUJUNI INNOCENT	**Integration, storage and synchronisation.** System architecture and boot order, node bus with sequence numbers, SD card gatekeeper and hash-chained log, occupancy ledger and NVS persistence, Wokwi diagram generator, report.	`main/main.c`, `bus/bus.c`, `storage/storage.c`, `services/occupancy.c`, `services/nvs_store.c`, `tools/gen_diagram.py`
MARTHA NAMUYIMBWA	[TODO: confirm] **Entrance access control.** MFRC522 RFID reader driver and reader task, access rules (anti-passback, schedule, lockdown, escort), door servos, card-scan flow.	`drivers/mfrc522.c`, `nodes/rfid.c`, `core_logic/access_policy.c`, `nodes/hub_access.c`, `nodes/door.c`
OCHEN FELIX	[TODO: confirm] **Lecture-room lighting.** Occupancy with grace period and dim warning, daylight harvesting, energy accounting, room node task, light sensor and lamp drivers.	`core_logic/room_logic.c`, `nodes/room_node.c`, `drivers/light_sensor.c`, `drivers/actuators.c`
NAKIMULI JOLLYNE FLAVIA	[TODO: confirm] **Management interfaces and authentication.** Serial command menu, two-factor admin login, LCD/buzzer user interface, schedule configuration.	`cli/*`, `core_logic/auth_fsm.c`, `services/auth.c`, `ui/ui.c`
TAMARA ALICE NAZZIWA	[TODO: confirm] **Testing and cloud dashboard.** Host unit tests, simulation test scenarios, Next.js dashboard with signed, idempotent event ingest.	`test/host/*`, `dashboard/*`

11.3 Individual video demonstrations

Each member recorded a screen video explaining the system and demonstrating the code and features listed against their name above.

#	Member	Video link
1	MUJUNI INNOCENT	**[TODO: video URL]**
2	MARTHA NAMUYIMBWA	**[TODO: video URL]**
3	OCHEN FELIX	**[TODO: video URL]**
4	NAKIMULI JOLLYNE FLAVIA	**[TODO: video URL]**
5	TAMARA ALICE NAZZIWA	**[TODO: video URL]**

A Serial Command Reference

Commands marked **(admin)** need an admin session (section 6.5); without one they reply with a hint to log in. Every configuration change is written to the log with the admin's card UID as `by=`.

Command	Effect
Monitoring	
`status`	Time, uptime, schedule and whether open now (minutes to closing), lockdown, emergency and escort state, people inside, granted/denied/alert counts, storage state, rejected bus frames, RFID reader health, admin session, and a line per room.
`who`	People inside: UID, name, role, entry time.
`rooms`	Per room: link, occupancy, lamp level, mode, lux, energy used and saved (Wh).
`users`	Registered cards: UID, name, role, enabled, has PIN, inside.
`log tail [n]`	The newest n log lines (default 10, at most 50).
`log verify [YYYYMMDD]`	Recompute the hash chain of a day's file (default today).
`log echo on|off`	Print every new log line live on the console.
`schedule`	Show the current access schedule.
`emergency`	Show the break-glass state.
`time`	Show the current time.
Session	
`login`	Start a login; then present the admin card at either reader.
`login <PIN>`	Second factor: the 4-digit PIN.
`logout`	End the admin session.
Configuration (admin)	
`schedule set HH:MM HH:MM [days]`	Set the access window; *days* is `weekdays`, `mon-sat`, `all` or a 7-letter mask such as `-MTWTF-` (Sunday first). Closing before opening means overnight.
`user add <UID> <name> <role> [PIN]`	Register a card (role `student`, `staff`, `security` or `admin`; admins need a PIN).
`user del|enable|disable <UID>`	Remove, enable or disable a card.
`light <1|2> auto|on|off`	Override a room's lighting, or return it to automatic.
`lockdown on|off`	Only security and admin may pass while on.
`emergency clear`	End a break-glass emergency and close the doors.
`time set YYYY-MM-DD HH:MM`	Set the clock and the DS1307 RTC.
Simulation	
`tap <UID> in|out`	Scan any card at a reader, e.g. `tap C0FFEE99 in` for an unregistered card.
`node <1|2> offline|online`	Cut or restore a room's link to show offline detection.
`help`	List all commands.

B Building, Running and Demonstrating

B.1 Build and run the firmware

```
. C:\Espressif\tools\Microsoft.v6.0.3.PowerShell_profile.ps1   # activate ESP-IDF v6.0.3
cd firmware
idf.py set-target esp32s3        # first time only
idf.py build
```

Then open `firmware/` in VS Code and run **Wokwi: Start Simulator** (it reads `wokwi.toml`). Click the serial monitor and type `help`. To present a card, click the ENTRY or EXIT reader, choose a card in its panel and press **Tap**. Key `e` presses the break-glass button.

B.2 Running the Wokwi link

Open `https://wokwi.com/projects/479125502548914177` and press play: wokwi.com builds the firmware from the project's source files with its online ESP-IDF builder (v5.3), chosen by `"builder": "esp-idf"` on the board in `diagram.json`, and starts the simulation. The online project holds the same sources as the repository, gathered into one folder (`firmware/wokwi/web/`, made by `make wokwi-web`), because the online builder has no components, CMake or Kconfig.

The code builds unchanged on both ESP-IDF versions: the project settings fall back to the defaults in `cocis_kconfig.h` when there is no Kconfig, `cocis_crypto.c` uses PSA Crypto on v6 and the classic mbedtls SHA-256 functions on v5, short waits do not depend on the FreeRTOS tick rate, and bring-up runs in its own task with its own stack.

A prebuilt copy of the firmware, `firmware/wokwi/cocis_smart_control.uf2.bin` (`make wokwi-fw`), can also be loaded into the project with **F1** (or right-click → **Command Palette**) → **Upload Firmware and Start Simulation...**; it lasts until the page is reloaded.

B.3 Other tasks (from the repository root)

```
make test       # host unit tests (gcc, or the gcc:14 Docker image)
make diagram    # regenerate firmware/diagram.json and the wiring figure
make pdf        # build this report -> docs/report.pdf (make report does the same)
make wokwi-web  # regenerate firmware/wokwi/web/, the files of the online Wokwi project
make wokwi-fw   # rebuild the prebuilt backup firmware/wokwi/cocis_smart_control.uf2.bin
make sim        # run the three Wokwi scenarios (needs wokwi-cli + WOKWI_CLI_TOKEN)
```

B.4 Demonstration script

The defaults are 07:00–22:00, Monday to Saturday, so run it during those hours. `ENTRY` and `EXIT` are the two readers.

```
status                          # overview, both RFID readers OK
<ENTRY: Tap Blue>               # Alice enters             -> Welcome Alice, entry door opens
<ENTRY: Tap Blue>               # again                    -> DENIED ANTI_PASSBACK
<EXIT: Tap Blue>                # Alice leaves             -> Goodbye Alice, exit door opens
<ENTRY: Tap Key fob>            # unregistered card        -> DENIED UNKNOWN_CARD
<Room 1: LDR below 150 lux, click PIR>   # occupied + dark -> lamp 100 %, LCD "R1 100% P-"
<wait about 20 s>               # dims to 30 %, then off at 30 s
<LDR above 400 lux, click PIR>  # bright                   -> lamp stays off, LCD shows S
<ENTRY: Tap Green, Tap Yellow>  # Bob and Carol enter
login                           # then on a reader: Tap Red (admin card), then:
```

```
login 1234
schedule set 07:00 08:00 all    # a window that has already ended today -> SHUT
<EXIT: Tap Green>               # Bob refused: OUTSIDE_HOURS + ALERT AFTER_HOURS_EXIT
<EXIT: Tap Yellow>              # Carol (security) leaves  -> escort window opens for 60 s
<EXIT: Tap Green>               # Bob leaves               -> OK_ESCORT
node 2 offline                  # 15 s later: ALERT NODE_OFFLINE, LCD "R2 offline"
node 2 online
<press e>                       # break glass: both doors open, alarm, EMRG
emergency clear
schedule set 07:00 22:00 mon-sat
who
log tail 20
log verify                      # OK: hash chain intact
```